

ASSIGNMENT-11.2

V.ARCHITHA

2306A91001

BATCH-30

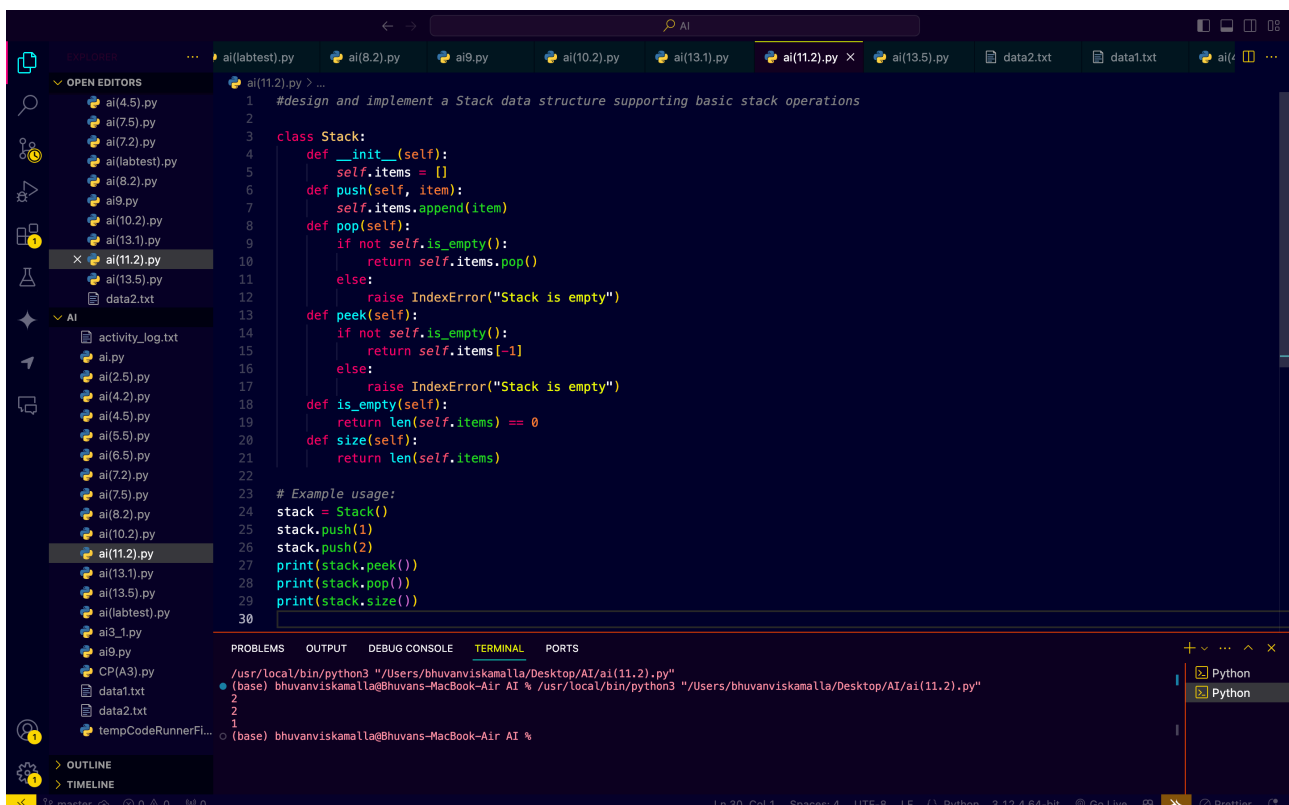
TASK-1:

PROMPT:

#Generate a python code to design and implement a Stack datastructure supporting basic stack operations.

A Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation.

CODE:



The screenshot shows a VS Code editor with a Python file named ai(11.2).py. The code implements a Stack class with the following methods: __init__ (initializes an empty list), push (appends an item), pop (removes and returns the last item, raising an IndexError if empty), peek (returns the last item without removing it, raising an IndexError if empty), is_empty (checks if the stack is empty), and size (returns the number of items). An example usage is provided at the bottom of the code block. The terminal at the bottom shows the command to run the script using Python 3.

```
1 #design and implement a Stack data structure supporting basic stack operations
2
3 class Stack:
4     def __init__(self):
5         self.items = []
6     def push(self, item):
7         self.items.append(item)
8     def pop(self):
9         if not self.is_empty():
10            return self.items.pop()
11        else:
12            raise IndexError("Stack is empty")
13     def peek(self):
14         if not self.is_empty():
15            return self.items[-1]
16        else:
17            raise IndexError("Stack is empty")
18     def is_empty(self):
19         return len(self.items) == 0
20     def size(self):
21         return len(self.items)
22
23 # Example usage:
24 stack = Stack()
25 stack.push(1)
26 stack.push(2)
27 print(stack.peek())
28 print(stack.pop())
29 print(stack.size())
30
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

/usr/local/bin/python3 "/Users/bhuvanviskamalla/Desktop/AI/ai(11.2).py"
(base) bhuvanviskamalla@Bhuvans-MacBook-Air AI % /usr/local/bin/python3 "/Users/bhuvanviskamalla/Desktop/AI/ai(11.2).py"
2
1
(base) bhuvanviskamalla@Bhuvans-MacBook-Air AI %

EXPECTED OUTPUT:

A Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation.

OBSERVATION :

- It is observed that in the out put will get by implementing FIFO principles
- The program successfully demonstrated stack and handled empty stack conditions

TASK-2:

PROMPT:

#Generate a python code to create a Queue data structure following FIFO principles

A complete Queue implementation including enqueue, dequeue, front element access, and size calculation

CODE:

The screenshot shows a VS Code editor with a Python file named `ai(11.2).py` open. The code implements a `Queue` class using a list to follow FIFO principles. The class methods include `__init__`, `enqueue`, `dequeue`, `front`, `is_empty`, and `size`. An example usage is provided at the bottom of the code block. The terminal at the bottom shows the command `python3 "/Users/bhuvanviskamalla/Desktop/AI/ai(11.2).py"` being executed. The file explorer on the left shows a project structure with various Python files and text files.

```
34
35 #create a queue data structure following FIFO principles ncluding enqueue, dequeue, front element access, and size calculation
36 class Queue:
37     def __init__(self):
38         self.items = []
39     def enqueue(self, item):
40         self.items.append(item)
41     def dequeue(self):
42         if not self.is_empty():
43             return self.items.pop(0)
44         else:
45             raise IndexError("Queue is empty")
46     def front(self):
47         if not self.is_empty():
48             return self.items[0]
49         else:
50             raise IndexError("Queue is empty")
51     def is_empty(self):
52         return len(self.items) == 0
53     def size(self):
54         return len(self.items)
55 # Example usage:
56 queue = Queue()
57 queue.enqueue(1)
58 queue.enqueue(2)
59 print(queue.front())
60 print(queue.dequeue())
61 print(queue.size())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(base) bhuvanviskamalla@Bhuvans-MacBook-Air AI % /usr/local/bin/python3 "/Users/bhuvanviskamalla/Desktop/AI/ai(11.2).py"

Ln 41, Col 23 Spaces: 4 UTF-8 LF Python 3.12.4 64-bit Go Live Prettier

Expected Output:

- A complete Queue implementation including enqueue, dequeue, front element access, and size calculation

OBSERVATION:

- it was able to implement the queue and dequeue ,font, size
- The program successfully demonstrated the queue operations

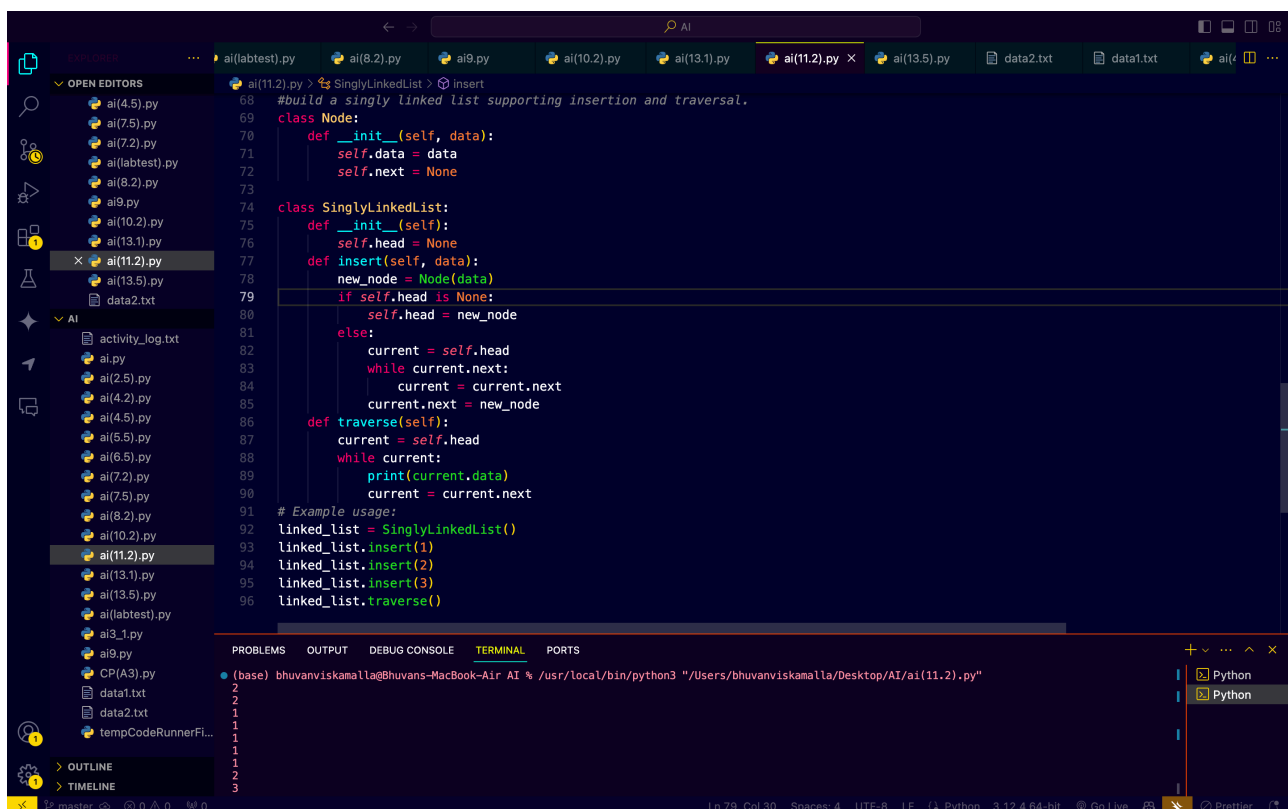
- TASK-3:

PROMPT:

Generate a python code build a singly linked list supporting insertion and traversal.

Correctly functioning linked list with node creation, insertion logic, and display functionality.

CODE:



```
ai(11.2).py > SinglyLinkedList > Insert
#build a singly linked list supporting insertion and traversal.
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None
    def insert(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node
    def traverse(self):
        current = self.head
        while current:
            print(current.data)
            current = current.next

# Example usage:
linked_list = SinglyLinkedList()
linked_list.insert(1)
linked_list.insert(2)
linked_list.insert(3)
linked_list.traverse()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(base) bhuvanviskamalla@Bhuvans-MacBook-Air AI % /usr/local/bin/python3 "/Users/bhuvanviskamalla/Desktop/AI/ai(11.2).py"

2
2
1
1
1
1
1
2
3

Ln 79, Col 30 Spaces: 4 UTF-8 LF Python 3.12.4 64-bit Go Live Prettier

Expected Output:

- Correctly functioning linked list with node creation, insertion logic, and display functionality.

OBSERVATION:

- it was able to perform all the linked list operations
- it was getting the accurate expected output by the generated example usage code.

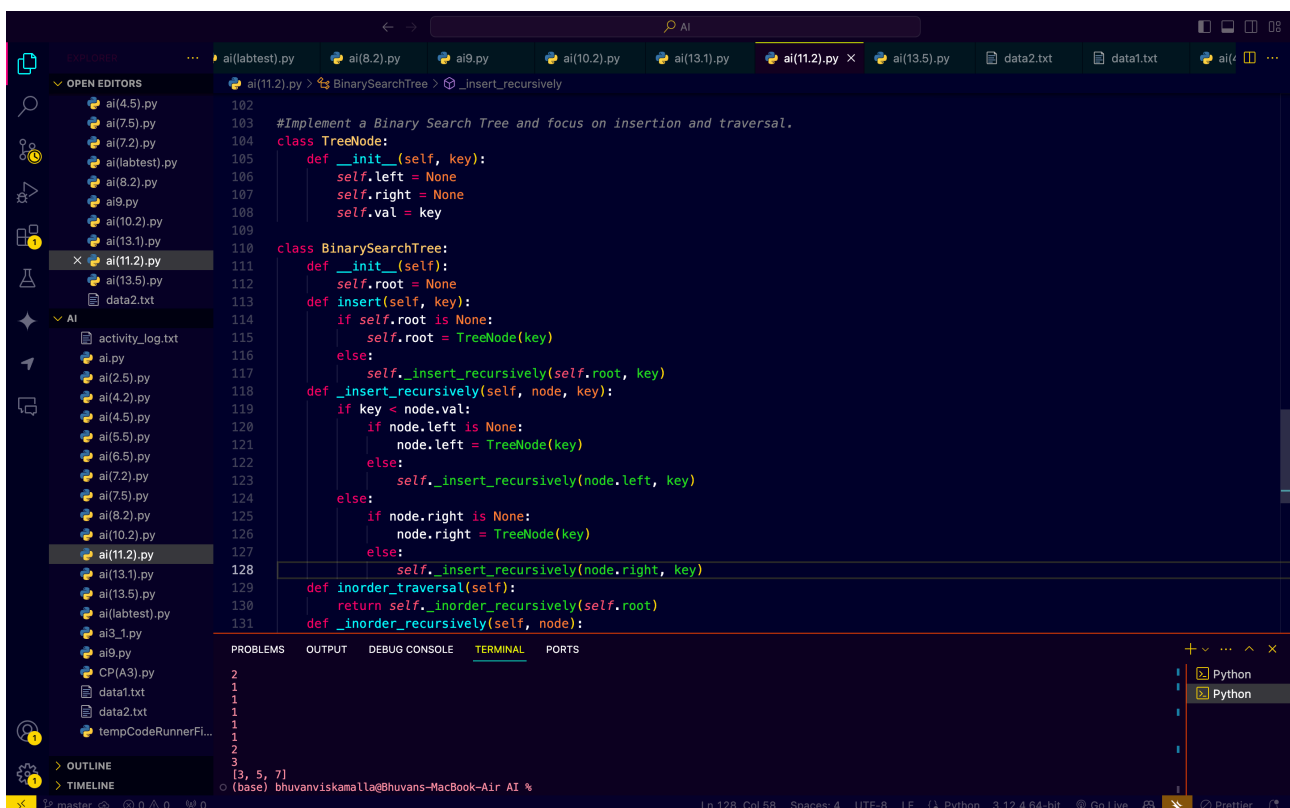
TASK-4:

PROMPT:

Generate a python code to get a Binary Search Tree with AI support focusing on insertion and traversal.

#BST program with correct node insertion and in-order traversal output.

CODE:



```
102
103 #Implement a Binary Search Tree and focus on insertion and traversal.
104 class TreeNode:
105     def __init__(self, key):
106         self.left = None
107         self.right = None
108         self.val = key
109
110 class BinarySearchTree:
111     def __init__(self):
112         self.root = None
113     def insert(self, key):
114         if self.root is None:
115             self.root = TreeNode(key)
116         else:
117             self._insert_recursively(self.root, key)
118     def _insert_recursively(self, node, key):
119         if key < node.val:
120             if node.left is None:
121                 node.left = TreeNode(key)
122             else:
123                 self._insert_recursively(node.left, key)
124         else:
125             if node.right is None:
126                 node.right = TreeNode(key)
127             else:
128                 self._insert_recursively(node.right, key)
129
130     def inorder_traversal(self):
131         return self._inorder_recursively(self.root)
132     def _inorder_recursively(self, node):
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

Expected Output:

- BST program with correct node insertion and in-order traversal

.

Output

OBSERVATION:

-it was able to perform all the BinarySearch Tree operations

-it was getting the accurate expected output by the generated example usage code.

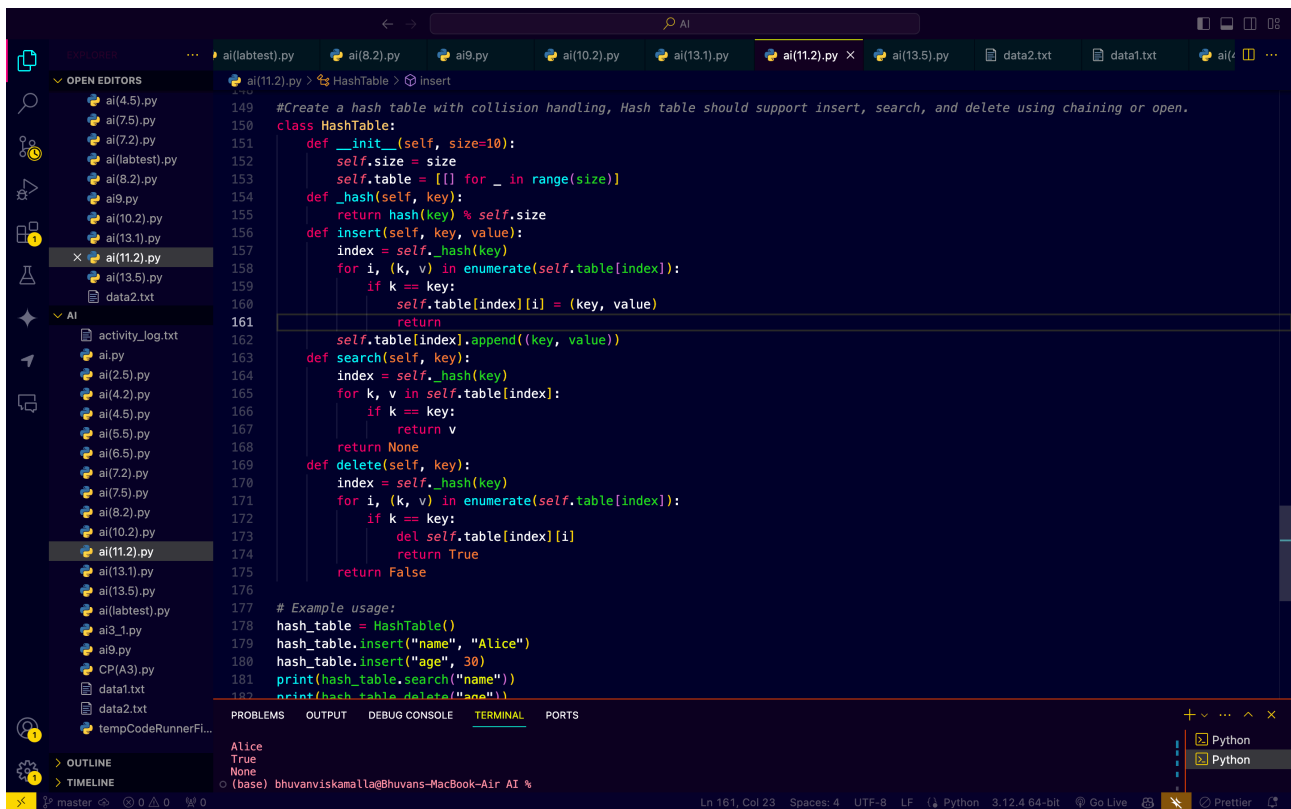
TASK-5

PROMPT:

Generate a python code to Create a hash table using AI with collision handling

#Hash table supporting insert, search, and delete using chaining
or open

CODE:



```
149 #Create a hash table with collision handling, Hash table should support insert, search, and delete using chaining or open.
150 class HashTable:
151     def __init__(self, size=10):
152         self.size = size
153         self.table = [[] for _ in range(size)]
154     def _hash(self, key):
155         return hash(key) % self.size
156     def insert(self, key, value):
157         index = self._hash(key)
158         for i, (k, v) in enumerate(self.table[index]):
159             if k == key:
160                 self.table[index][i] = (key, value)
161                 return
162         self.table[index].append((key, value))
163     def search(self, key):
164         index = self._hash(key)
165         for k, v in self.table[index]:
166             if k == key:
167                 return v
168         return None
169     def delete(self, key):
170         index = self._hash(key)
171         for i, (k, v) in enumerate(self.table[index]):
172             if k == key:
173                 del self.table[index][i]
174                 return True
175         return False
176
177 # Example usage:
178 hash_table = HashTable()
179 hash_table.insert("name", "Alice")
180 hash_table.insert("age", 30)
181 print(hash_table.search("name"))
182 print(hash_table.delete("name"))
```

Terminal Output:

```
Alice
True
None
```

Expected Output:

- Hash table supporting insert, search, and delete using chaining or open

OBSERVATION:

- it was able to perform all the Hash Table operations
- it was getting the accurate expected output by the generated example usage code.